

Multi-tenancy data model — 99 tables: 90 tenant-scoped, 9 global

GLOBAL — 9 tables

No `tenant_id`; shared by all tenants

Tenants

`tenant`

Roles & permissions (catalogue)

`role`
`permission`
`role_permission`
`user_role`

Auth tokens

`refresh_token`
`password_reset_token`

National HESA specs

`statutory_spec_version`
`statutory_advisory`

Why global: the tenant list itself, a shared role / permission catalogue, login tokens looked up before the tenant is known, and national HESA specifications that are the same for every institution.

LEGEND

Solid box: tenant-scoped (has `tenant_id`, RLS on)

Dashed box: global (no `tenant_id`)

* has `tenant_id` but RLS not yet enabled

Every tenant-scoped table: `tenant_id` `uuid` → `tenant.id` (FK, ON DELETE RESTRICT), indexed, nullable today (NOT NULL pending), stamped automatically on insert. Primary keys stay a single `uuid` id — `tenant_id` is NOT part of the primary key.

tenant

— one row per institution (global)

`id` `uuid` PK
`name` `text`
`subdomain` `text` UNIQUE (`icr` → `icr.<domain>`)
`activated_at` / `deactivated_at`
`branding` `json`

`tenant_id` → `tenant.id`

TENANT-SCOPED — 90 tables (every row carries `tenant_id`)

People & identity 5 tables

`person`, `person_contact`,
`person_relationship`,
`person_merge_record`, `users*`

Recruitment & admissions 8 tables

`application`, `application_assessment`,
`candidate_stage_history`, `interview`,
`interview_panellist`, `offer`,
`offer_condition`, `reference_request`

Research & opportunities 6 tables

`research_opportunity`,
`research_demand`, `research_area`,
`research_award`, `research_project`,
`project_code`

Student record 7 tables

`programme`, `department`, `student`,
`student_lifecycle_event`,
`student_custom_field`,
`student_custom_value`, `cost_centre`

Taught 7 tables

`taught_module`, `module_offering`,
`module_enrolment`, `module_assessment`,
`assessment_result`, `dissertation`,
`taught_award`

Supervision & progression 8 tables

`supervisor_relationship`,
`supervision_meeting`,
`supervision_commitment`,
`milestone_definition`, `milestone`,
`progression_review`,
`progression_appeal`,
`review_panel_member`

Thesis & completion 6 tables

`thesis`, `thesis_correction`,
`examination`, `examiner_nomination`,
`completion`, `award`

Funding 4 tables

`funding_source`, `funding_arrangement`,
`stipend_payment`, `fee_waiver`

Statutory & exports 4 tables

`report_profile`,
`report_field_mapping`, `export_job`,
`change_finding`

Workflow & messaging 9 tables

`workflow_definition`,
`workflow_instance`, `task`,
`notification`,
`notification_preference`,
`outbox_event`, `integration_log`,
`email_bounce`, `portal_message`

Documents 4 tables

`document`, `document_version`,
`document_chunk`, `evidence_claim`

Settings, policy & audit 5 tables

`institution_setting`,
`value_set_override`, `policy_version`,
`policy_proposal`, `audit_log`

AI, intelligence & ML 17 tables

`composer_run*`,
`assistant_unmatched_query`,
`assistant_write_intent`,
`engagement_event`, `engagement_snapshot`,
`intervention_plan`

`intervention_action`,
`intervention_outcome`,
`intelligence_telemetry`,
`driver_snapshot`, `prediction_snapshot`,
`ml_dataset`

`ml_finding`, `ml_model`,
`ml_model_version`, `ml_prediction`,
`ml_training_run`

How the rows are separated

student (one shared table, both institutions)

id (PK)	tenant_id	student_ref	name
7f3a...	ICR ...a91	ICR-0001	A. Patel
b20c...	Neuro ...c44	NE-0001	J. Smith
19de...	ICR ...a91	ICR-0002	L. Chen
e84f...	Neuro ...c44	NE-0002	R. Ahmed



Request on icr.<domain> → SET LOCAL app.current_tenant = ...a91

id (PK)	tenant_id	student_ref	name
7f3a...	ICR ...a91	ICR-0001	A. Patel
19de...	ICR ...a91	ICR-0002	L. Chen



Request on neuro.<domain> → SET LOCAL app.current_tenant = ...c44

id (PK)	tenant_id	student_ref	name
b20c...	Neuro ...c44	NE-0001	J. Smith
e84f...	Neuro ...c44	NE-0002	R. Ahmed

Same query — `SELECT * FROM student` — in both requests. PostgreSQL's row-level security policy removes the other institution's rows before the app sees them.

What makes it work

Piece	How it works	Where
Column	Each tenant-scoped table has <code>tenant_id uuid → tenant.id (FK, ON DELETE RESTRICT)</code> plus an index. Added by <code>TenantMixin</code> .	<code>app/db/base.py</code>
Stamping	New rows get the request's tenant automatically on insert (default from the request context), so code never sets it by hand.	<code>TenantMixin</code> default
Tenant per request	Subdomain (<code>icr.<domain></code>) and the login token's tenant claim must agree; a mismatch is refused (403). The tenant is then set with <code>SET LOCAL app.current_tenant</code> for that transaction, on read and write sessions.	<code>tenant_resolver.py</code> , sessions
RLS policy	<code>ENABLE + FORCE</code> row-level security on each scoped table, one policy: <code>tenant_id = NULLIF(current_setting('app.current_tenant', true), '')::uuid</code> (unset → bypass for migrations, seeds and the worker).	<code>app/db/tenant_ddl.py</code>
Fail-closed role	Optional <code>pgr_app</code> database role for the API: if the tenant isn't set, it sees zero rows instead of all rows.	<code>deploy/provision_app_role.sql</code>
Unique keys	Where a value must be unique per institution, the constraint includes <code>tenant_id</code> (e.g. custom attribute key: unique on (<code>tenant_id</code> , <code>key</code>)). Primary keys stay a single <code>uuid</code> .	<code>model __table_args__</code>

Open items: `tenant_id` is still nullable (flip to `NOT NULL` once every write path is confirmed to stamp it) · `users` and `composer_run` have `tenant_id` but row-level security isn't enabled on them yet · the fail-closed role is opt-in (`APP_DATABASE_URL`); recommended for production.